

fb2d Instruction Set Architecture (raw opcode fetch fork)

62 opcodes + NOP. Every 16-bit cell value is a valid simulator state. The IP extracts the 11 data bits **without Hamming correction**, then uses the opcode decoder to select an instruction. The five parity bits do not participate in instruction selection.

This fork retains the v1.15 instruction set: I (syndrome inspect) replaces the unused M (payload subtract via IX) at opcode 54. M was completely superseded by m (raw XOR) in the copy-down architecture. The change is the fetch rule, not the 16-bit cell format or the semantics of the individual instructions.

Cell Format

Each cell stores 16 bits in the following Hamming(16,11) SECDED layout. All 65,536 patterns are allowed; exactly 2,048 have valid parity:

Bit: 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
d10 d9 d8 d7 d6 d5 d4 p3 d3 d2 d1 p2 d0 p1 p0 p_all

- **Payload:** 11 data bits → opcode or value (0-2047)
- **Parity:** 5 check bits at positions 0, 1, 2, 4, 8
- Arithmetic ops (+, -, ., *, /, %, P, Q) maintain the Hamming invariant
- Bit-level ops (r, l, R, L, Y, x, z, f, m, j, l) operate on all 16 raw bits

Instruction Selection

Forward and reverse stepping use the same read rule:

16-bit cell → raw 11-bit payload → opcode lookup → instruction

In fb2d.py, the lookup is `_PAYLOAD_TO_OPCODE[_CELL_TO_PAYLOAD_RAW[cell]]`. Neither this read nor the lookup changes the stored cell. The CLI and GUI instruction labels use this same interpretation. Parity remains available to instructions that explicitly read or modify it.

Opcode Encoding

Opcode payloads are drawn from an [11,6,4] linear code with minimum Hamming distance $d_{\min} = 4$. This means no combination of 1, 2, or 3 data-bit flips can turn one designated opcode payload into another designated payload. The numeric opcode IDs used in the instruction tables below are not their stored payloads: `OPCODE_PAYLOADS` in fb2d.py supplies the mapping (for example, opcode 15, +, has payload 975).

Nearest-codeword decoding: each 11-bit payload within Hamming distance 1 of a designated non-NOP opcode payload selects that opcode. Starting from a designated encoding, one data-bit flip preserves the instruction and two data-bit flips give NOP. Three or more data-bit flips can select a different instruction. Parity-bit flips have no effect on this choice. Each non-NOP opcode accepts 12 payloads (one center and 11 single-bit neighbors), so 744 of 2,048 payloads select non-NOP instructions and the remaining 1,304 select NOP.

These error guarantees concern distance from a designated instruction payload, not from any Hamming-valid data word. A correct parity check alone does not make a payload a designated instruction encoding.

Why Fetch Does Not Perform Hamming Correction

The former fetch path applied Hamming decoding before the opcode lookup. That could move a payload across an opcode boundary. For example, flipping cell bits 0, 1, and 6 in a designated + encoding changes its raw payload from 975 to 971. Raw opcode decoding still selects +, since 971 is one data bit from 975. Hamming decoding instead produces payload 963, two bits from 975, and the subsequent opcode lookup selects NOP.

The fork avoids that interaction by using only raw payload extraction during fetch. This does not claim better survival for every noise pattern.

Other Reads and Explicit Repair

Use	Interpretation
Instruction fetch, forward and reverse	Raw payload followed by opcode lookup
Payload arithmetic and CL/EX zero tests	Raw data bits; no Hamming decoding
R, L, Y rotation amounts	Hamming-decoded payload of [CL], then & 15
I, V	Explicit Hamming syndrome/parity operations on raw 16-bit values
Raw swaps, XORs, and rotated data	Full 16-bit values, including parity positions

Hamming(16,11) has minimum distance 4. Its checks enable single-bit correction and double-bit detection across all 16 positions. The agent uses I to inspect cells, V to form a single-error mask, and j to write the repair. Double-bit errors need extra information; the current agent consults the corresponding cell in its partner gadget. Three or more errors can imitate the single- or double-error cases.

Reversibility does not require Hamming decoding during fetch: the stored instruction bits are retained, both time directions use the same lookup, and the existing overlap guards protect the inputs needed to undo each instruction. The repair program preserves error information in its auxiliary cells and waste rather than discarding it.

Heads

Head	Purpose
H0	Primary data head
H1	Secondary data head
IX	Interceptor — programmable head for cross-gadget correction
CL	Condition latch — tested by conditional mirrors; also provides rotation amounts
EX	Exteroceptor — breadcrumb trail for reversibility

All heads point to cells on the same toroidal grid as the IP.

Notation

- $[H0]$ = full 16-bit cell value at head H0's position
- $\text{payload}(X)$ = the 11 data bits extracted from cell X without correction
- $\text{decoded_payload}(X)$ = the 11-bit result of the Hamming decoder; used for rotation amounts only among the instruction operands below
- Movement is on a torus: edges wrap

Mirrors (6 opcodes)

Op	Opcode	Description
/	1	Unconditional / reflect
\	2	Unconditional \ reflect
%	3	/ reflect if $\text{payload}([CL]) \neq 0$
?	4	/ reflect if $\text{payload}([CL]) == 0$
&	5	\ reflect if $\text{payload}([CL]) \neq 0$
!	6	\ reflect if $\text{payload}([CL]) == 0$

Mirror geometry: / maps $E \leftrightarrow N$ and $S \leftrightarrow W$; \ maps $E \leftrightarrow S$ and $N \leftrightarrow W$.

Conditional mirrors test whether any data bit in [CL] is set ($[CL] \& \text{DATA_MASK}$). Parity bits are ignored.

Head Movement (20 opcodes)

Op	Opcode	Description
N	7	Move H0 north
S	8	Move H0 south
E	9	Move H0 east
W	10	Move H0 west
n	11	Move H1 north
s	12	Move H1 south
e	13	Move H1 east
w	14	Move H1 west
>	23	Move CL east
<	24	Move CL west
^	25	Move CL north
v	26	Move CL south
]	29	Move EX east
[	30	Move EX west
}	31	Move EX south
{	32	Move EX north
H	49	Move IX north
h	50	Move IX south
a	51	Move IX east
d	52	Move IX west

All movement wraps on the torus.

Byte-Level Data (8 opcodes)

These operate on Hamming-encoded cells, maintaining the parity invariant.

Op	Opcode	Description	Inverse
+	15	payload([H0])++ (mod 2048, with parity fixup)	−
−	16	payload([H0])− (mod 2048, with parity fixup)	+
.	17	payload([H0]) += payload([H1]) (mod 2048, with parity fixup)	,
,	18	payload([H0]) -= payload([H1]) (mod 2048, with parity fixup)	.
X	19	swap([H0], [H1]) — full 16-bit swap	self
F	20	if payload([CL]) != 0: swap([H0], [H1]) — Fredkin gate	self
G	21	swap(H1 register, [H0]) — indirect H1	self
T	22	swap([CL], [H0]) — bridge (full 16-bit)	self

Arithmetic ops (+, −, ., ,) XOR a precomputed delta into the cell so that both the payload and parity bits update together. The operation is bijective on all 65536 cell values, not just valid codewords.

F (Fredkin gate) tests $\text{payload}([CL]) \neq 0$ — any nonzero data bit.

Bit-Level Data (10 opcodes)

These operate on the full 16-bit raw cell value (including parity bits). They do **not** maintain the Hamming invariant — by design, the correction gadget uses them to manipulate raw bits.

Op	Opcode	Description	Inverse
x	39	[H0] ^= [H1] — XOR	self
r	40	[H0] rotate right 1 bit	l
l	41	[H0] rotate left 1 bit	r
f	42	if [CL] & 1: swap([H0], [H1]) — bit-0 Fredkin	self
z	43	swap(bit 0 of [H0], bit 0 of [H1])	self
R	44	[H0] rotate right by (decoded_payload([CL]) & 15) bits	L
L	45	[H0] rotate left by (decoded_payload([CL]) & 15) bits	R
Y	46	[H0] ^= ror([H1], decoded_payload([CL]) & 15) — fused rotate-XOR	self
:	47	payload([CL])++ (mod 2048, with parity fixup)	;
;	48	payload([CL])– (mod 2048, with parity fixup)	:

Key distinctions:

- **f** (lowercase) gates on raw **bit 0** of [CL] — used by the barrel-shifter correction gadget
- **F** (uppercase) gates on **payload([CL]) != 0** — used for general conditional logic
- **R/L/Y** read the rotation amount from `decoded_payload([CL]) & 15` (0-15 positions). This Hamming decoding is an operand rule, not opcode selection, and does not write a correction back to CL.
- **z** swaps raw bit 0 of [H0] with raw bit 0 of [H1] — used to extract single syndrome/parity bits

EX (Exteroceptor) Ops (8 opcodes)

Op	Opcode	Description	Inverse
P	27	payload([EX])++ — leave breadcrumb	Q
Q	28	payload([EX])– — erase breadcrumb	P
K	33	swap(CL register, EX register) — exchange head positions	self
(	34	\ reflect if payload([EX]) != 0	—
)	35	\ reflect if payload([EX]) == 0	—
#	36	/ reflect if payload([EX]) == 0	—
\$	37	/ reflect if payload([EX]) != 0	—
Z	38	swap([H0], [EX]) — full 16-bit EX swap	self

EX mirrors test payload([EX]) (data bits only, ignoring parity).

The standard loop pattern is (P ... %:

1. (tests [EX]: if payload != 0, reflect (skip loop body). On first entry [EX] is 0, so IP continues into the body.
2. P increments [EX], leaving a breadcrumb (makes [EX] nonzero).
3. Loop body executes.
4. % tests [CL]: if payload([CL]) != 0, reflect back toward (.

On re-entry, (sees nonzero [EX] and reflects (exits). The reverse path uses Q to erase the breadcrumb, restoring [EX] to 0.

IX Interceptor Ops (4 opcodes)

IX is a programmable interceptor for cross-gadget correction. In the dual-gadget architecture, each gadget's IX points at the other gadget's code cells. The copy-down pattern: *m* copies a remote codeword to a local cell, correction runs locally, then *j* writes the correction mask back to the remote cell.

Op	Opcode	Description	Inverse
<i>m</i>	53	$[H0] \wedge= [IX]$ — raw 16-bit XOR (copy-in / uncompute)	self
<i>I</i>	54	$[H0] \wedge= \text{syndrome_5bit}([IX])$ — syndrome inspect	self
<i>j</i>	55	$[IX] \wedge= [H0]$ — raw 16-bit write-back	self
<i>V</i>	56	$[H0] \wedge= (1 \ll \text{syndrome_4bit}([IX]))$ — correction mask	self

m, *I*, *j*, and *V* all operate on raw 16-bit cell values (like bit-level ops).

I (syndrome inspect, v1.15): Computes the 4-bit Hamming syndrome AND overall parity of $[IX]$, maps all 5 result bits to `DATA_MASK` positions, and XORs them into $[H0]$:

- $s0 \rightarrow \text{bit } 3 (d0)$, $s1 \rightarrow \text{bit } 5 (d1)$, $s2 \rightarrow \text{bit } 6 (d2)$, $s3 \rightarrow \text{bit } 7 (d3)$
- $p_all \rightarrow \text{bit } 9 (d4)$

Starting with $[H0] = 0$, the result has `payload([H0]) != 0` exactly when $[IX]$ fails at least one parity check, including bit-0-only errors ($\text{syndrome}=0$, $p_all=1$). Passing all checks does not prove that the cell still contains the original value: some larger error patterns produce another valid codeword. Self-inverse (XOR). NOP when $H0 == IX$.

I enables a **pre-syndrome filter**: before copying in the remote cell, test its integrity with *I* T ?. Cells with valid parity ($\text{syndrome}=0$, $p_all=0$) bypass the entire correction pipeline in ~8 ops instead of ~70. Cells with failed checks continue to the full correction path. This replaces 123 ops of Phase C syndrome computation on the bypass path with a single opcode.

The copy-down pattern for correcting a remote cell:

1. *m* — copy remote codeword $[IX]$ into local zero cell via XOR
2. Run correction logic locally (*H0*, *H1*, *CL*, *EX* on the stomach row)
3. *m* — uncompute local copy (XOR again zeroes the cell)
4. *j* — write correction mask back: $[IX] \wedge= [H0]$

Only *IX* touches remote rows; all other heads stay local.

V (correction mask, v1.15): Computes $1 \ll \text{hamming_syndrome_4bit}([IX])$ and XORs it into $[H0]$. In standard-form Hamming(16,11), the 4-bit syndrome IS the error bit position, so $1 \ll \text{syndrome}$ is the exact single-bit correction mask. Self-inverse (XOR). NOP when $H0 == IX$.

V replaces Phases C+D+C' (~160 ops of syndrome computation, barrel shifting, and uncompute) with a single opcode. Assuming at most two flipped bits per scanned cell, the pre-syndrome filter and overall-parity probe route single-bit errors to *V*, double-bit errors to copy-over, and clean cells to bypass. Larger error patterns can be misclassified. Shrinks the gadget from 379 to 147 ops.

IX Horizontal Momentum Ops (3 opcodes)

IX momentum ops give *IX* a persistent horizontal direction (*ix_dir*, per-IP, defaults to East). This enables serpentine scanning: *IX* sweeps east across a row, detects a boundary, then retreats, moves vertically, flips direction, and sweeps west — systematic row-by-row coverage without coprimality constraints.

Op	Opcode	Description	Inverse
<i>A</i>	57	Advance <i>IX</i> one step in <i>ix_dir</i>	<i>B</i>
<i>B</i>	58	Retreat <i>IX</i> one step opposite <i>ix_dir</i>	<i>A</i>
<i>U</i>	59	Flip <i>ix_dir</i> via XOR 2 ($E \leftrightarrow W$, $N \leftrightarrow S$)	self

The horizontal boundary detection pattern (local-only, no remote write):

1. A — advance IX in current direction
2. m — copy [IX] to local [H0] via XOR (H0 was 0)
3. T — move value to CL for testing
4. ? — if [CL]==0 (zero cell = boundary), redirect to handler
5. T — restore CL, m — restore H0 (on non-boundary path)
6. Handler: / B C U : \ — retreat, advance vertically, flip, signal

IX Vertical Momentum Ops (3 opcodes)

Per-IP field `ix_vdir` (defaults to South). Enables ping-pong bounded scanning: after a horizontal boundary, C advances IX vertically. A vertical boundary test detects when IX leaves the code+handler area. On vertical boundary: D 0 C bounces IX back (retreat, flip, re-advance). IX ping-pongs between the first code row and handler row without entering stomach/waste rows.

Op	Opcode	Description	Inverse
C	60	Advance IX one step in <code>ix_vdir</code>	D
D	61	Retreat IX one step opposite <code>ix_vdir</code>	C
0	62	Flip <code>ix_vdir</code> via XOR 2 (N↔S, E↔W)	self

Vertical boundary test pattern (on last code row, between merge gates):

```
T Z ]      deposit handler signal to waste row
m T ? T m   test [IX] after vertical advance
```

If [IX]=0 (outside code+handler area): ? fires, IP drops to bounce sub-handler: / D 0 C : \ (retreat, flip `ix_vdir`, re-advance, signal).

Full ping-pong scan (5 rows, W=99): rows 7→8→9→10→11→10→9→8→7→8... ~864 cycles per full down-up sweep.

Reversibility Pairs

Every opcode has a unique inverse (itself or another opcode):

Forward	Inverse	Notes
+	−	Increment / decrement
.	,	Add / subtract
r	l	Rotate right / left by 1
R	L	Rotate right / left by CL
:	;	CL increment / decrement
P	Q	EX breadcrumb / erase
N/S	S/N	H0 movement pairs
E/W	W/E	H0 movement pairs
n/s	s/n	H1 movement pairs
e/w	w/e	H1 movement pairs
>/<	</>	CL movement pairs
^/v	v/^	CL movement pairs
] / [	[/]	EX movement pairs
} / {	{ / }	EX movement pairs
H/h	h/H	IX movement pairs
a/d	d/a	IX movement pairs
A/B	B/A	IX horizontal momentum advance / retreat
C/D	D/C	IX vertical momentum advance / retreat

Self-inverse ops: X, F, G, T, K, Z, x, f, z, Y, m, I, j, V, U, 0

Mirrors (/, \, %, ?, &, !, (,), #, \$) are self-inverse: the IP direction mapping is its own inverse — running the IP backward through a mirror reflects it the same way.

NOP

Any payload not corresponding to opcodes 1-62 (and not within Hamming distance 1 of such a payload) is a NOP: the IP advances without side effects. The canonical “empty cell” is raw value 0 (payload 0).

Special Cell Values

Two non-opcode cell values have architectural significance:

Symbol	Payload	Raw Value	Description
o	1017	0x7E8E	NOP filler. The 64th (unused) codeword of the [11,6,4] opcode code. As a true codeword, it has $d_{\min}=4$ from all opcode codewords: all 1-bit AND 2-bit data errors still decode to NOP (0/55 two-bit pairs produce a real opcode). Used for padding on code rows, bypass rows, return rows, and handler rows. Data-bit distance 8 from zero (robust boundary detection).
~	2047	0xFFFF	Boundary marker. All bits set. Decodes to NOP (not within Hamming distance 1 of any opcode). Used for IX scan boundary rows (top and bottom) and boundary columns (col 0 and col W-1). Detected via $m \ T : ? ;$ $T \ m \text{ —} :$ wraps payload 2047→0, $?$ fires on zero. Enables agents in non-zero environments where zero cells are not reliably empty.

Reference Exports

docs/isa.md is the editable source. Rebuild the HTML, plain-text, and PDF references after changing it:

```
pandoc docs/isa.md --standalone --mathml \
  --metadata title="fb2d ISA: raw opcode fetch" -o docs/isa.html
pandoc docs/isa.md --to=plain --wrap=auto --columns=100 \
  -o docs/isa.text
pandoc docs/isa.md --pdf-engine=xelatex \
  --include-in-header=docs/isa-pdf-header.tex \
  -V geometry:margin=0.65in -V fontsize=10pt \
  -V mainfont="Arial" -V monofont="Menlo" -o docs/isa.pdf
```

The PDF command uses fonts available on macOS; use equivalent local fonts on other systems. These references describe the fork; historical design notes describe earlier implementations.